Software Testing / Assessment

Name: يوسف السيد محمد عاشور

Section: مسجل المادة من فرقة ثالثة

Q1. Black-Box/EP testing

Analysis

The method Sailing.available(boolean weekend, boolean july, int windforce) determines whether sailing is available based on three paramters. The specification states:

weekend - true if a weekend
july - true if in the month of July
windforce - on the standard scale (0-12)
Return: true if sailing today (windforce <5, weekend or any day in July), otherwise false
Throws: Exception - any invalid inputs

Equivalence Partitions and Boundary Values

Input: windforce (int)

Valid partition VP1: [0, 1, 2, 3, 4] - Sailing possible (windforce < 5)
Valid partition VP2: [5, 6, 7, 8, 9, 10, 11, 12] - Too windy (windforce ≥ 5)
Invalid partition IP1: < 0 - Below valid range (throws Exception)
Invalid partition IP2: > 12 - Above valid range (throws Exception)

Boundary values: -1, 0, 4, 5, 12, 13
Input: weekend (boolean)
Partition 1: true (it is a weekend)
Partition 2: false (it is a weekday)

Input: july (boolean)
Partition 1: true (it is July - ship sails every day)
Partition 2: false (other months - ship only sails on weekends)

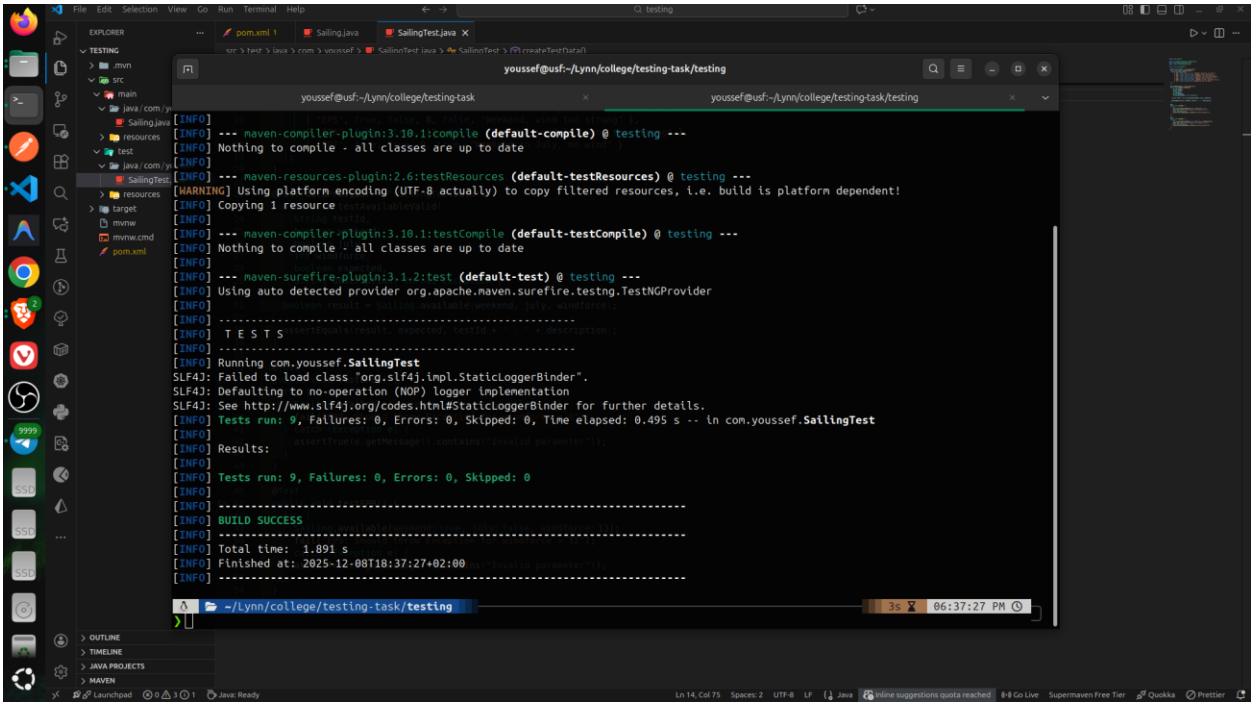
Test Cases

Test ID	weekend	july	windforce	Expected Output	Test Data Rationale
EP1	**true**	**false**	**2**	**true**	**Weekend, not July, good wind (VP1, weekend=T)**
EP2	**false**	**true**	**3**	**true**	**Weekday in July, good wind (VP1, july = T)**
EP3	**false**	**false**	**4**	**false**	**Weekday, not July (VP1, but no valid day)**
EP4	true	true	**5**	**false**	Weekend in July, wind too strong (VP2, boundary)
EP5	**true**	**false**	**8**	**false**	Weekend, wind too strong (VP2, middle value)
EP6	**false**	**true**	**12**	**false**	July, maximum valid windforce (VP2, boundary)
EP7	**true**	**true**	**0**	**true**	Weekend in July, minimum windforce (VP1, boundary)
EP8	**false**	**false**	**-1**	**Exception**	Invalid windforce below range (IP1, boundary)
EP9	**true**	**talse**	**13**	**Exception**	Invalid windforce above range (IP2, boundary)

Test Implementation (Java/TestNG code)

```
package com.youssef;

import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;
import static org.testng.Assert.*;

public class SailingTest {

@DataProvider(name = "sailingTestData")
public Object[][] createTestData() {
return new Object[][] {
{ "EP1", true, false, 2, true, "Weekend, good wind, not July" },
{ "EP2", false, true, 3, true, "Weekday in July, good wind" },
{ "EP3", false, false, 4, false, "Weekday, not July, good wind" },
{ "EP4", true, true, 5, false, "Weekend in July, wind too strong" },
{ "EP5", true, false, 8, false, "Weekend, wind too strong" },
{ "EP6", false, true, 12, false, "Weekday in July, wind too strong" },
{ "EP7", true, true, 0, true, "Weekend in July, no wind" }
};
}

@Test(dataProvider = "sailingTestData")
public void testAvailableValid(
String testId,
boolean weekend,
boolean july,
int windforce,
boolean expected,
String description) throws Exception {

boolean result = Sailing.available(weekend, july, windforce);

assertEquals(result, expected, testId + " : " + description);
}

@Test
public void testEP8() {
try {
Sailing.available(false, false, -1);
fail("EP8: Should throw Exception for windforce < 0");
} catch (Exception e) {
assertTrue(e.getMessage().contains("Invalid parameter"));
}
}

@Test
public void testEP9() {
try {
Sailing.available(true, false, 13);
fail("EP9: Should throw Exception for windforce > 12");
} catch (Exception e) {
assertTrue(e.getMessage().contains("Invalid parameter"));
}
}
}
```

Test Results Screenshot



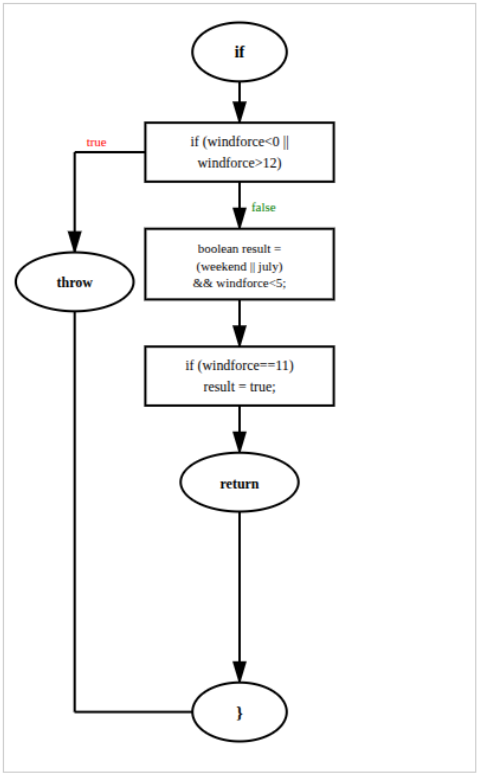
Q2. White-Box/SC testing

Control Flow Graph (CFG) for Sailing.available()

The CFG consists of the following nodes:

Node 1 (Entry): Method entry point - "if"
Node 2: Parameter validation check (line 16-18)
Node 3 (throw): Throw Exception for invalid parameters
Node 4: Main logic - evaluate weekend/july conditions (line 19-21)
Node 5: Check if windforce==11 (lines 22-23)
Node 6 (return): Return result (line 25)
Node 7 (Exit): Method exit - "}"

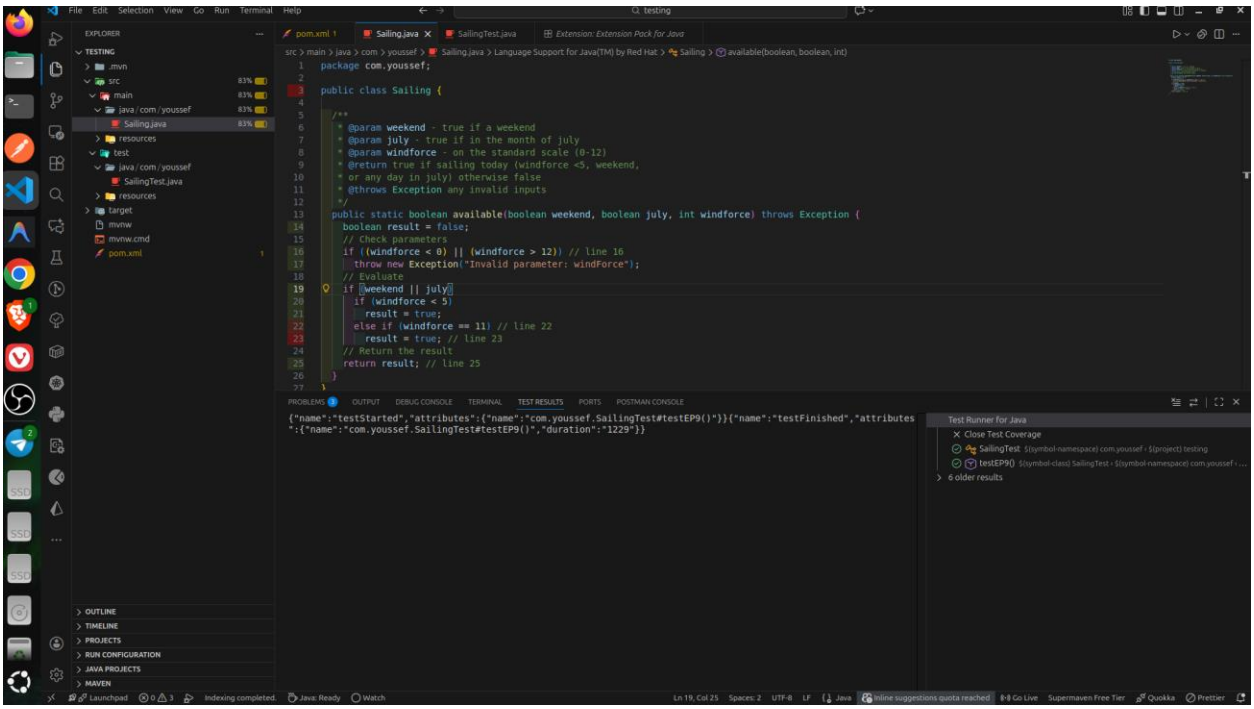
Note: An extra node is needed in the CFG to represent the final '}' – the throw statement should result in a branch to this final '}' node. The return statement should also be followed by an edge to the final '}' node.



Analysis - Coverage from EP Tests

Using EclEmma to measure statement coverage of `Sailing.available()` by the EP tests:

Coverage Results BEFORE adding SC tests



Missing Nodes and Required Test Data

Nodes NOT covered by EP tests:

Lines 22-23: The condition `if (windforce==11)` and its body `result = true;`

Data values required to execute missing nodes:

windforce = 11: This specific value triggers the special condition on lines 22-23
Any values for weekend and july (since this is checked after the main logic)

Additional Test Cases for 100% Statement Coverage

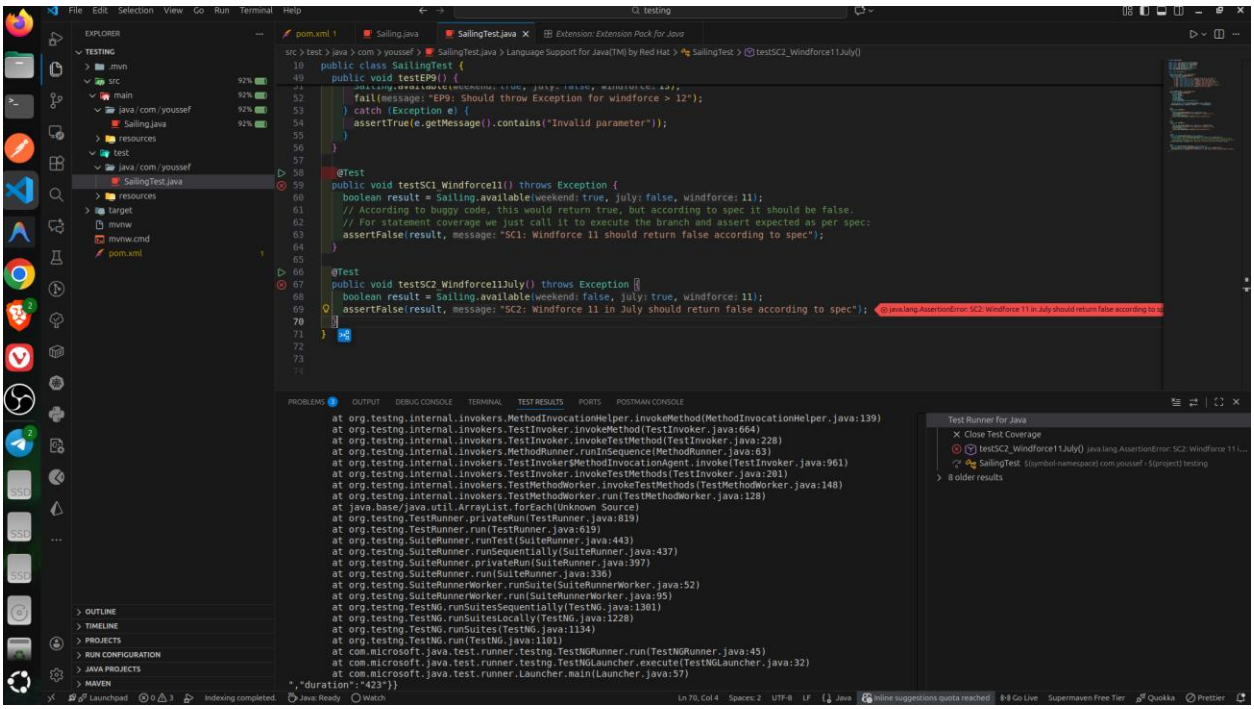
Test ID	weekend	july	windforce	Expected Output	Coverage Target

SC1	true	false	11	false	Cover lines 22-23 (windforce==11 condition)
SC2	false	true	11	false	Alternative path to lines 22-23

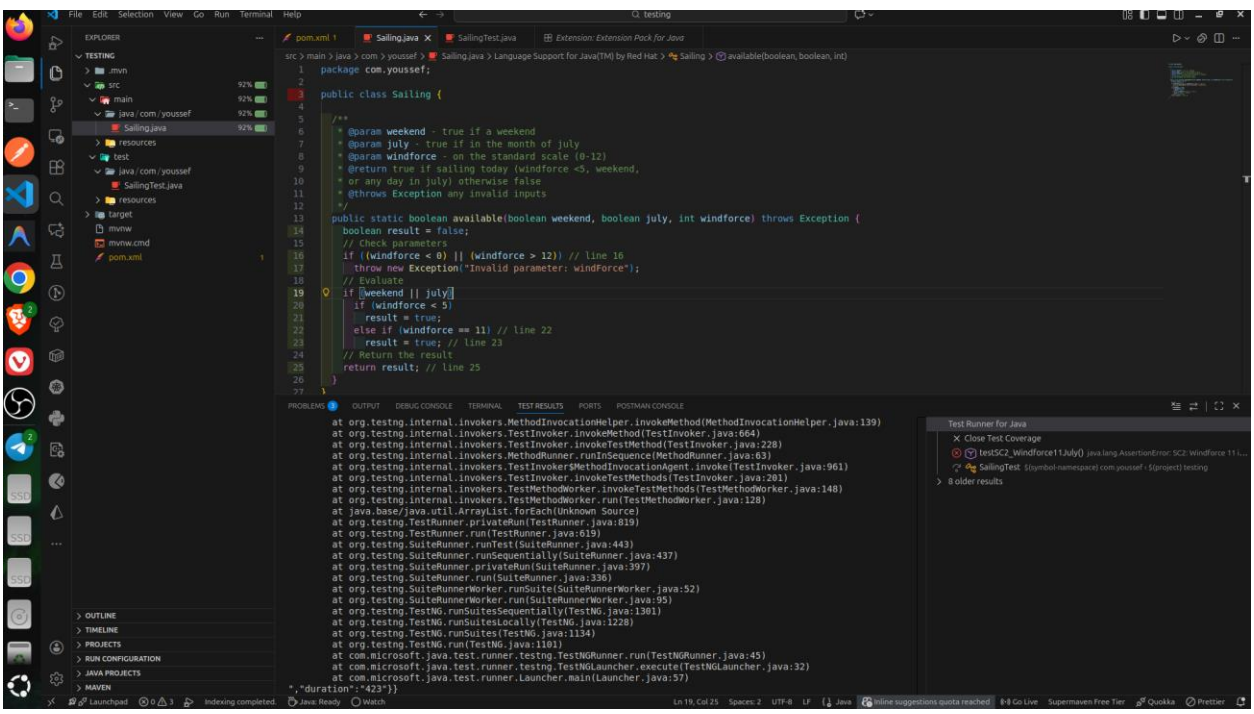
Test Implementation (Java/TestNG code)

```
@Test
public void testSC1_Windforce11() throws Exception {
boolean result = Sailing.available(true, false, 11);
// According to buggy code, this would return true, but according to spec it should be false.
// For statement coverage we just call it to execute the branch and assert expected as per spec:
assertFalse(result, "SC1: Windforce 11 should return false according to spec");
}

@Test
public void testSC2_Windforce11July() throws Exception {
boolean result = Sailing.available(false, true, 11);
assertFalse(result, "SC2: Windforce 11 in July should return false according to spec");
}
```

Test Results Screenshot - SHOWING A FAILURE



Coverage Results AFTER adding SC tests (100% Statement Coverage)



Q3. Black-Box/Combinational testing (using truth-tables)

Analysis - Causes and Effects

Causes (Input Conditions):

C1: windforce < 5 (true or false)
C2: weekend (true or false)
C3: july (true or false)

Effects (Outputs):

E1: Return true (sailing available)
E2: Return false (sailing not available)

Hint applied: Using the single rule "0<=windforce<=4" means we consider windforce values 01234 as representatives for C1=true, and values 56789101112 for C1=false.

Full Truth Table (8 candidate rules)

Rule	C1: windforce<5	C2: weekend	C3: july	E1: true	E2: false
1	T	T	T	X	
2	T	T	F	X	
3	T	F	T	X	
4	T	F	F		X
5	F	T	T		X
6	F	T	F		X
7	F	F	T		X
8	F	F	F		X

Simplified Truth Table (4 final rules using don't care conditions)

By applying don't care conditions, we can reduce the number of rules:

Rules 1 & 2 can be combined: If windforce<5 AND weekend=T, july doesn't matter → true
Rules 5, 6, 7, 8 can be combined: If windforce≥5, both weekend and july don't matter → false

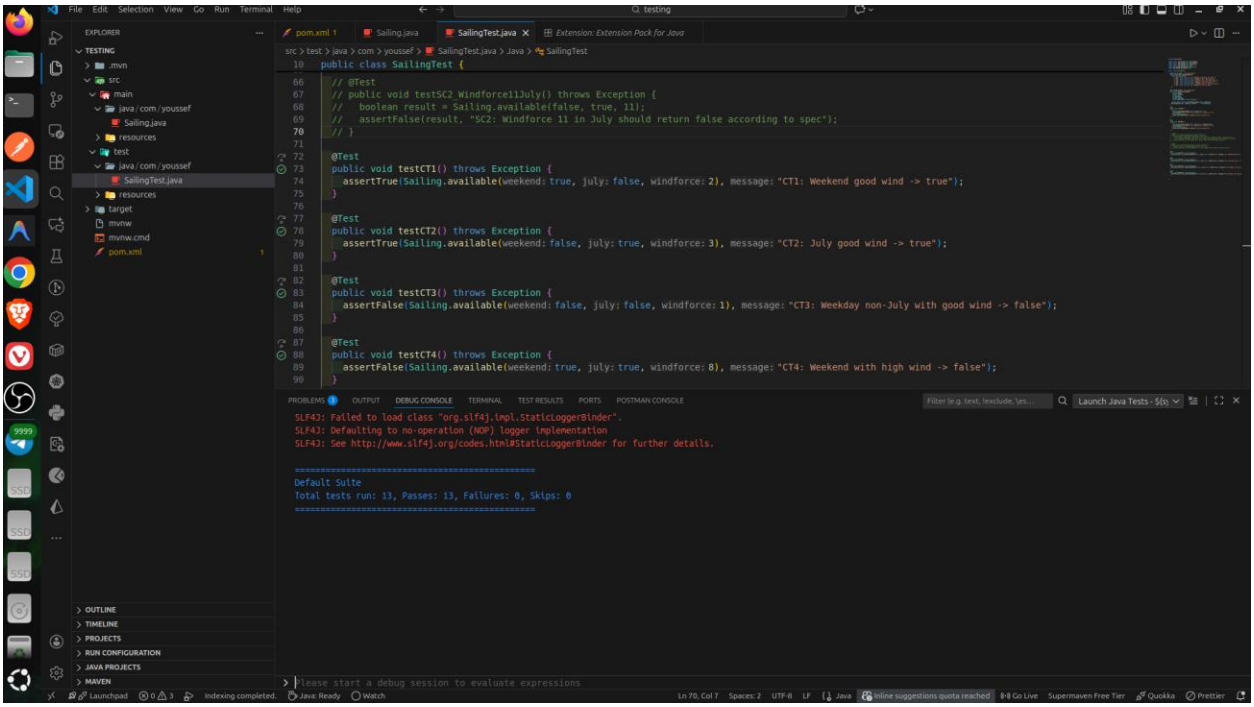
Rule	C1: windforce<5	C2: weekend	C3: july	E1: true	E2: false
1	T	T	-	X	
2	T	F	T	X	
3	T	F	F		X
4	F	-	-		X

Test Cases (One per simplified rule)

Test ID	weekend	july	windforce	Expected output	Rule Covered
CT1	true	false	2	true	Rule 1 (weekend=T, windforce<5)
CT2	false	true	3	true	Rule 2 (weekday, July, windforce<5)
CT3	fasle	false	1	false	Rule 3 (weekday, not July, windforce<5)
CT4	true	true	8	false	Rule 4 (windforce≥5)

Test Implementation (Java/TestNG code)

```
@Test
public void testCT1() throws Exception {
assertTrue(Sailing.available(true, false, 2), "CT1: Weekend good wind -> true");
}

@Test
public void testCT2() throws Exception {
assertTrue(Sailing.available(false, true, 3), "CT2: July good wind -> true");
}

@Test
public void testCT3() throws Exception {
assertFalse(Sailing.available(false, false, 1), "CT3: Weekday non-July with good wind -> false");
}

@Test
public void testCT4() throws Exception {
assertFalse(Sailing.available(true, true, 8), "CT4: Weekend with high wind -> false");
}
```


File Edit Selection View Go Run Terminal Help
testing
EXPLORER
TESTING
.mvn
src
main
java / com / youssef
Sailing.java
resources
test
java / com / youssef
SailingTest.java
resources
target
mvnw
mvnw.cmd
pom.xml
pom.xml
Sailing.java
SailingTest.java
Extension: Extension Pack for Java
src > test > java > com > youssef > SailingTest.java > Java > SailingTest
public class SailingTest {
// @Test
// public void testSC2_Windforce11July() throws Exception {
// boolean result = Sailing.available(false, true, 11);
// assertFalse(result, "SC2: Windforce 11 in July should return false according to spec");
// }
@Test
public void testCT1() throws Exception {
assertTrue(Sailing.available(weekend: true, july: false, windforce: 2), message: "CT1: Weekend good wind -> true");
}
@Test
public void testCT2() throws Exception {
assertTrue(Sailing.available(weekend: false, july: true, windforce: 3), message: "CT2: July good wind -> true");
}
@Test
public void testCT3() throws Exception {
assertFalse(Sailing.available(weekend: false, july: false, windforce: 1), message: "CT3: Weekday non-July with good wind -> false");
}
@Test
public void testCT4() throws Exception {
assertFalse(Sailing.available(weekend: true, july: true, windforce: 8), message: "CT4: Weekend with high wind -> false");
}
PROBLEMS 3
OUTPUT
DEBUG CONSOLE
TERMINAL
TEST RESULTS
PORTS
POSTMAN CONSOLE
Launch Java Tests - ${p
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
==========
Default Suite
Total tests run: 13, Passes: 13, Failures: 0, Skips: 0
==========
OUTLINE
TIMELINE
PROJECTS
RUN CONFIGURATION
JAVA PROJECTS
MAVEN
Please start a debug session to evaluate expressions
Launchpad
Indexing completed.
Java: Ready
Watch
Ln 70, Col 7
Spaces: 2
UTF-8
LF
Java
Inline suggestions quota reached
Go Live
Supermaven Free Tier
Quokka
Prettier

Q4. Black-Box/random testing

Analysis - Decision Tree for Random Test Generation

To generate random test data that produces a specific expected output, I use the following decision tree:

1. Randomly select expected output: random(true, false)
2. If expected output = **true**:
    a. windforce = random(0..4) // Must be < 5
    b. weekend = random(true, false)
    c. If weekend = true:
        - july = random(true, false) // Doesn't matter
    d. If weekend = false:
        - july = true // Must be July
3. If expected output = **false**:
    a. Choose randomly between two cases:
    b. Case 1 (bad wind):
        - windforce = random(5..12) // Wind too strong
        - weekend = random(true, false) // Doesn't matter
        - july = random(true, false) // Doesn't matter
    c. Case 2 (wrong day with good wind):
        - windforce = random(0..4) // Good wind
        - weekend = false // Weekday
        - july = false // Not July

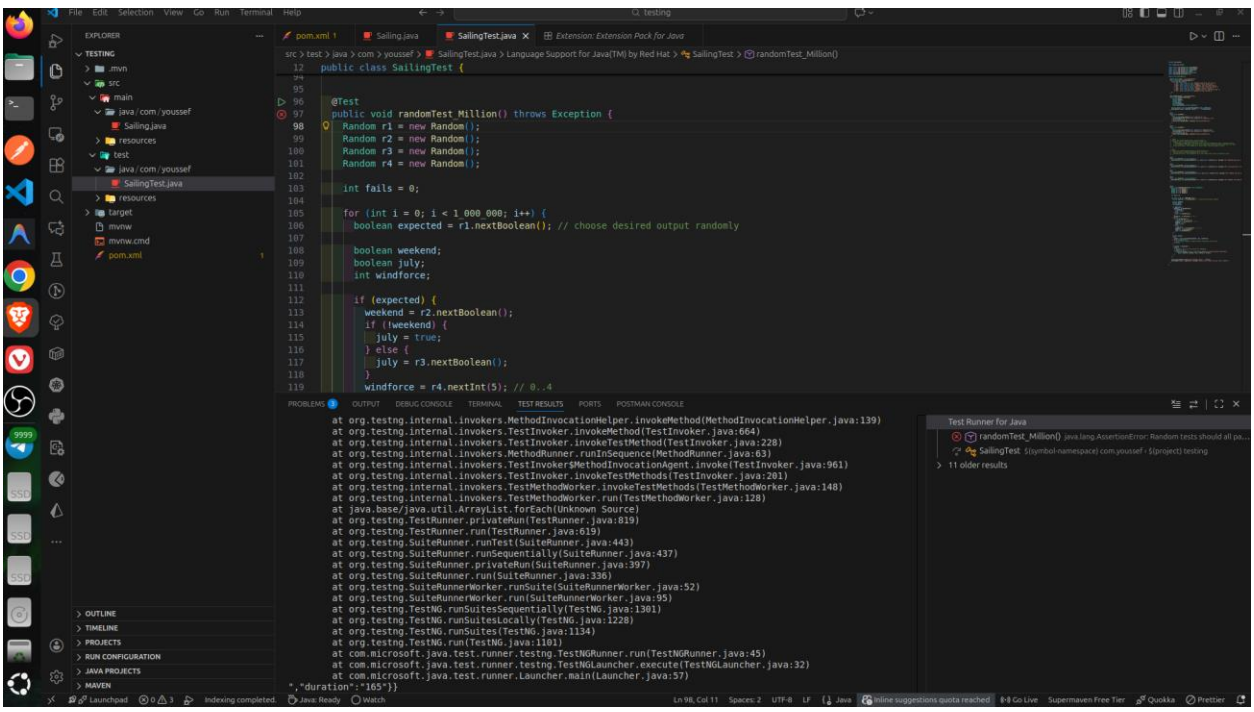
Test Implementation (Java/TestNG code)

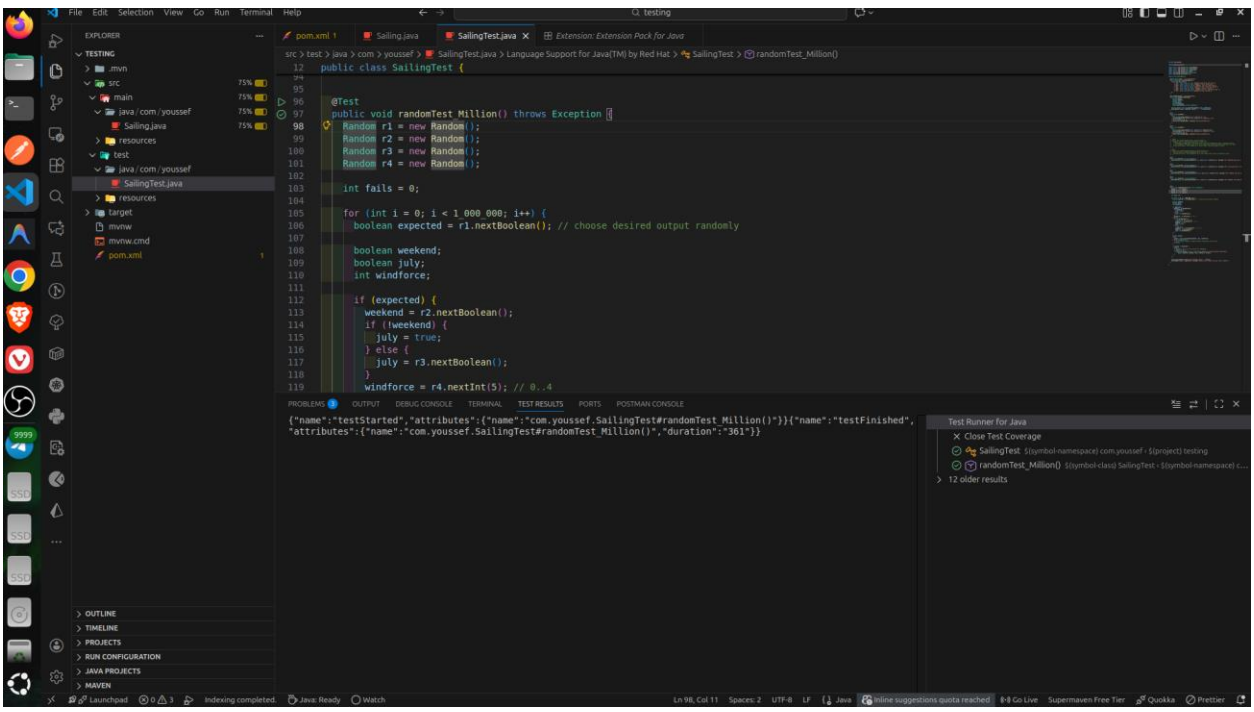
```
@Test
public void randomTest_Million() throws Exception {
Random r1 = new Random();
Random r2 = new Random();
Random r3 = new Random();
Random r4 = new Random();
```

```
int fails = 0;

for (int i = 0; i < 1_000_000; i++) {
boolean expected = r1.nextBoolean(); // choose desired output randomly

boolean weekend;
boolean july;
int windforce;

if (expected) {
weekend = r2.nextBoolean();
if (!weekend) {
july = true;
} else {
july = r3.nextBoolean();
}
windforce = r4.nextInt(5); // 0..4
} else {
if (r2.nextBoolean()) {
// good wind but wrong day
windforce = r4.nextInt(5); // 0..4
weekend = false;
july = false;
} else {
// bad wind
windforce = 5 + r4.nextInt(8); // 5..12
weekend = r2.nextBoolean();
july = r3.nextBoolean();
}
}

boolean actual;
try {
actual = Sailing.available(weekend, july, windforce);
} catch (Exception e) {
// skip invalid (shouldn't happen because we generate valid values)
continue;
}

if (actual != expected) {
fails++;
// Optional: log first few failures for debugging
```

```
if (fails <= 5) {
System.out.printf("Fail %d: expected=%b weekend=%b july=%b wind=%d actual=%b%n",
fails, expected, weekend, july, windforce, actual);
}
}
}

System.out.println("Random test finished. Fails: " + fails);
assertEquals(fails, 0, "Random tests should all pass (fails found)");
}
```

Test Results - WITH FAULT (before commenting lines 22-23)



Test Results - FAULT REMOVED (after commenting lines 22-23)

Now comment out lines 22 and 23 in Sailing.java to remove the fault, and re-run the random test.